Chapter 12. Hands-On Projects

Now that you've gained a fundamental grasp of financial data engineering, it's time to put it into practice. In this chapter, you will go through a series of practical projects designed to give you firsthand experience working with financial data.

Four projects will be discussed, each focusing on a different problem and employing a unique technological stack:

1. Constructing a bank account management system with PostgreSQL
2. Building a financial data ETL workflow with Mage
3. Developing a financial microservice workflow with Netflix Conductor
4. Implementing a reference data store with OpenFIGI, PermID, and GLEIF APIs

A few points should be mentioned about these projects. First, they are meant to provide hands-on experience with financial data engineering and may not necessarily represent complete solutions for real business problems. Additionally, they are intended to be executed locally on your machine and not deployed in a production environment. Finally, the employed technologies are not indicative of the author's personal preferences; instead, they reflect the author's prudent consideration of what would best clarify the problems and solutions for the reader.

Prerequisites

All projects in this chapter will be packaged and isolated with all their dependencies into Docker containers. Docker is the most popular open source software for operating system (OS) virtualization (aka containerization). It is available for Windows, macOS, and Linux through *Docker*

Desktop. Therefore, to run the projects on your machine, you must install the Docker Desktop version that is compatible with your operating system. Please follow the instructions in the official Docker documentation to complete the installation. Furthermore, as you will be running more than one container in each project, you will be using a specific multicontainer orchestration tool called Docker Compose, which is ideal for local testing and development.

The other prerequisite is to clone this book's GitHub repository onto your local machine. First, make sure you have Git installed. Then, open a terminal in your computer, navigate to the location where you want to pull the project files, and finally, clone the repository.

Project 1: Designing a Bank Account Management System Database with PostgreSQL

Account management is a core functionality for banks, allowing them to handle and oversee customer accounts, balances, and transactions. In this project, you will design and implement a relational database system for managing bank accounts. You will be using the conceptual/logical/physical data modeling approach discussed in Chapter 8.

Conceptual Model: Business Requirements

In the conceptual phase, the focus is on understanding business requirements and defining the high-level structure of the database system. Operational and business models vary from one banking institution to another, and not all banks offer the same products and services. As such, you will design a simple and generic bank account management system for this project.

We'll assume that stakeholders have defined their requirements and that these have been formalized them into entities, relationships, and con-

straints. Let's explore each in greater depth.

Entities

Our bank account management system needs to store data for seven types of entities: accounts, customers, loans, transactions, branches, employees, and cards.

Accounts are the most essential product that banks offer to their customers. For this reason, the account entity is needed to maintain detailed records of the various account types offered by the bank (e.g., savings, checking), along with account IDs, associated customers, balances, and more.

Loans are also a key banking product. Thus, the loan entity is needed to store information such as loan ID, terms (including amount, duration, interest rate, payment schedule, and start and end dates), type (such as mortgage or personal loan), and other relevant details.

In addition to accounts and loans, most banks provide a range of payment cards to their clients. As a result, the card entity is crucial for storing card-related information, including cardholder ID, associated accounts, card numbers, issuance and expiration dates, and other relevant details.

Customers represent the bank's client base, including both individuals and organizations. Therefore, the customer entity is essential for storing vital client information, such as IDs, names, addresses, statuses (e.g., active, inactive), and contact details (e.g., email addresses and phone numbers).

Employees are the individuals hired by the bank to deliver various services to customers. To manage employee information, the employee entity is required to store attributes such as employee IDs, names, job titles, and the branch where they are assigned.

Branches are the bank's physical locations where customers engage with employees to perform various transactions. The branch entity is essential

for managing branch information, including branch IDs, names, addresses, and phone numbers.

Lastly, banks must track all transactions associated with customer accounts. Therefore, the transaction entity is vital for recording transaction details such as transaction IDs, associated account IDs, employee IDs (for transactions initiated by employees), timestamps, and amounts.

Relationships

The business team has requested that the following relationships be established between entities:

- Each account should be linked to a customer.
- Each loan should be linked to a customer.
- Each loan payment should be linked to a transaction and an account.
- Each employee should be affiliated with a branch.
- Each card should be associated with both a customer and an account.

These relationships ensure data integrity and facilitate efficient data retrieval and management.

Constraints

The business team has outlined the following constraints to be enforced within the database implementation:

- Account balances must never go below a customer-specific minimum amount.
- All entity records (e.g., accounts, loans, transactions) must be identified with a unique ID.
- Data redundancy should be minimized.
- Null values are not permitted for certain fields.
- Specific fields, such as email addresses, must be unique across records.

Constraints play a pivotal role in ensuring high data quality, consistency, integrity, and compliance, which in turn contributes to the reliability of

the account management system.

Logical Model: Entity Relationship Diagram

Now that we have stakeholder agreement on the account management conceptual model, the next step is to build the logical model. This stage focuses on selecting the data storage model most suitable for our system (a concept discussed in Chapter 8).

After a thorough evaluation, the financial data engineering team has concluded that the relational model is the best fit. This model effectively organizes various entities into distinct tables and ensures data integrity through database constraints. Moreover, the relational model supports the implementation of a normalized data structure, which is essential for our system.

To implement this, we need to create the *Entity Relationship Diagram* (ERD) for our logical model. An ERD is a visual representation used to model the structure of a database. It illustrates the entities (such as tables or objects) within a system, their attributes (such as fields or properties), and the relationships among these entities. An ERD is constructed using information collected and summarized from the previous conceptual phase. Various tools are available for drawing ERDs, including Lucidchart, Creately, DBDiagram, ERDPlus, DrawSQL, QuickDBD, and EdrawMax. Figure 12-1 illustrates the ERD of our account management system.

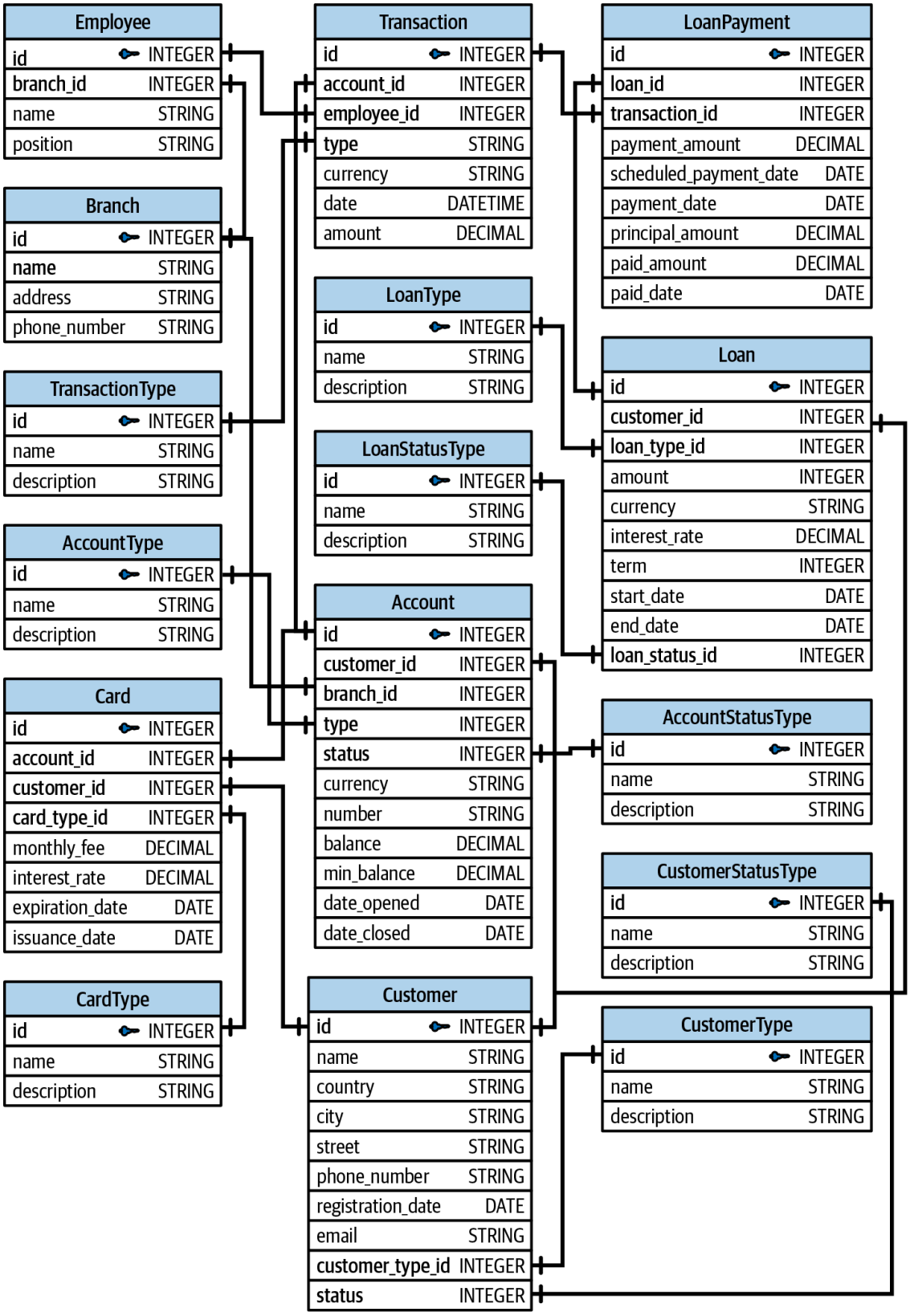

Figure 12-1. Entity Relationship Diagram of our example bank account management system

With the ERD design complete, let's move on to the next step: selecting the database technology and translating our ERD into database queries.

Physical Model: Data Definition and Manipulation

Language

With our logical data model ready, it's time to choose a relational database technology and write the queries to create and populate our tables. Let's assume the financial data engineering team has selected PostgreSQL as the database management system due to its high reliability and strong adherence to SQL standards.

Now, let's shift gears and test things on your machine.

Project 1: Local Testing

To interact with PostgreSQL, you will need to run two Docker containers: one for the PostgreSQL database instance and another for pgAdmin, a user-friendly UI for interacting with PostgreSQL.

To launch the two containers, open a terminal and navigate to the following path:

```
# Bash
cd {path/to/book/repo}/FinancialDataEngineering/book/chapter_12/project_1
```

Then execute the following `docker compose` command to run our project containers:

```
# Bash
docker compose up -d
```

After that, open a browser and paste the URL *http://localhost:8080/* into a new tab. Wait until you see the pgAdmin UI and log in using the dummy credentials (PGADMIN_DEFAULT_EMAIL and PGADMIN_DEFAULT_PASSWORD) in the *.env* file. Remember that this setting is for local testing purposes, and you should never share or store passwords publicly or explicitly in your project files.

Once logged in, click on Add New Server and insert the following values:

- In the General tab, give a name to the server (e.g., Bank Account).
- In the Connections tab, set:
  - Host name/address = *postgres*
  - Port = *5432*
  - Maintenance database = *bank_account*
  - Username = *admin*
  - Password = *password*

These are the values that you set in the project's environmental variables file *.env*. In the upcoming projects, we'll utilize pgAdmin, and you'll need to configure it by following similar steps. However, you'll need to use the credentials provided in the *.env* file specific to each project. Once completed, click on Save. Figure 12-2 illustrates these steps.

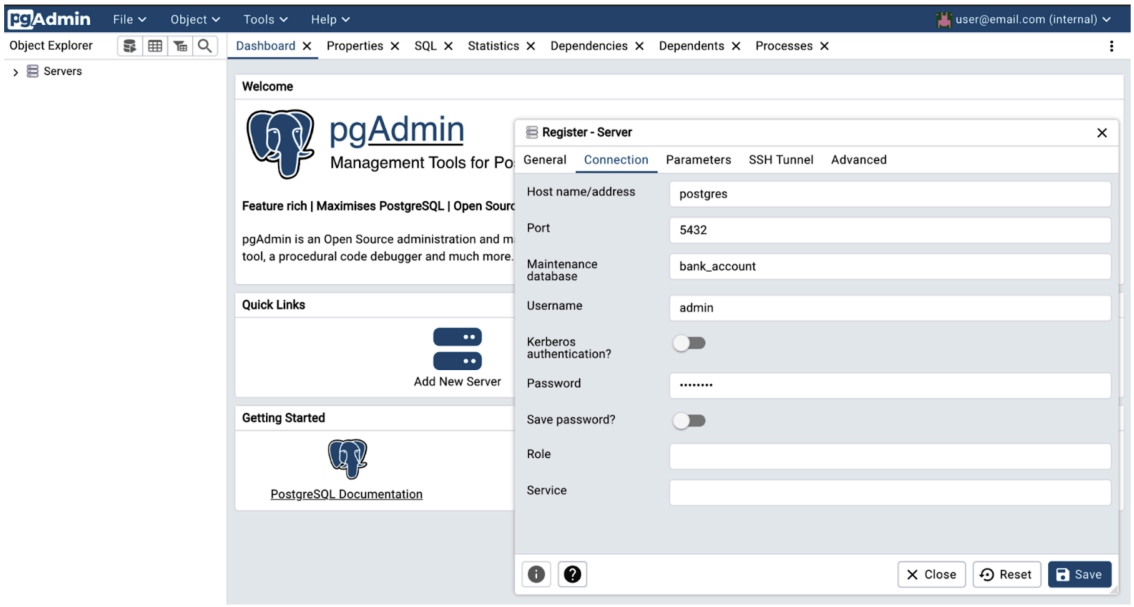

Figure 12-2. Creating a new server in pgAdmin

After that, from the left sidebar, follow these steps (as shown in Figure 12-3):

1. Navigate to Servers → Bank Account → Databases → bank_account → Schemas → public → Tables.
2. Right-click on Tables and select Query Tool.

Now, we are ready to create our tables. In SQL, using the term *Data Definition Language* (DDL) to denote statements that create or alter database objects is common. DDL statements include `CREATE`, `ALTER`, and

`DROP`. The full list of table creation queries is available on this book's GitHub repo. You will need to copy and paste all the queries into the Query Tool and click the Run arrow on the top, as illustrated in Figure 12-3. This will create all the tables for our bank account management system. To see the tables, right-click Tables from the left sidebar and hit Refresh. Then you will be able to see the 16 tables, as illustrated in Figure 12-4.

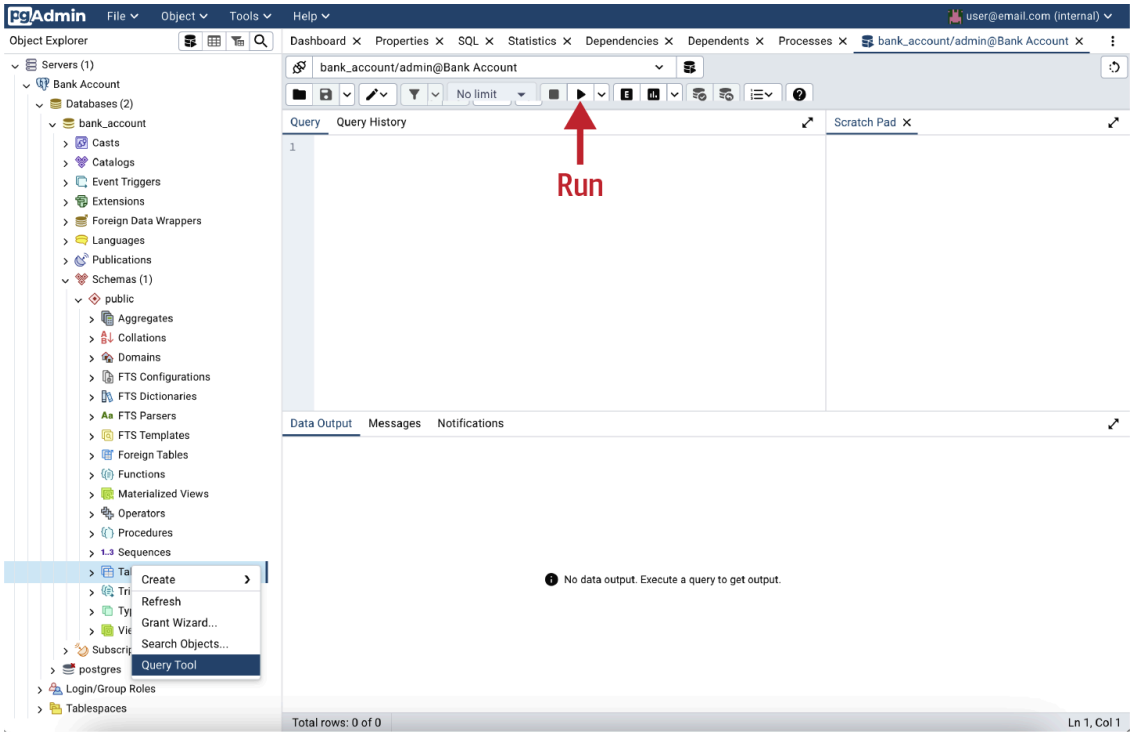

Figure 12-3. Opening the Query Tool editor

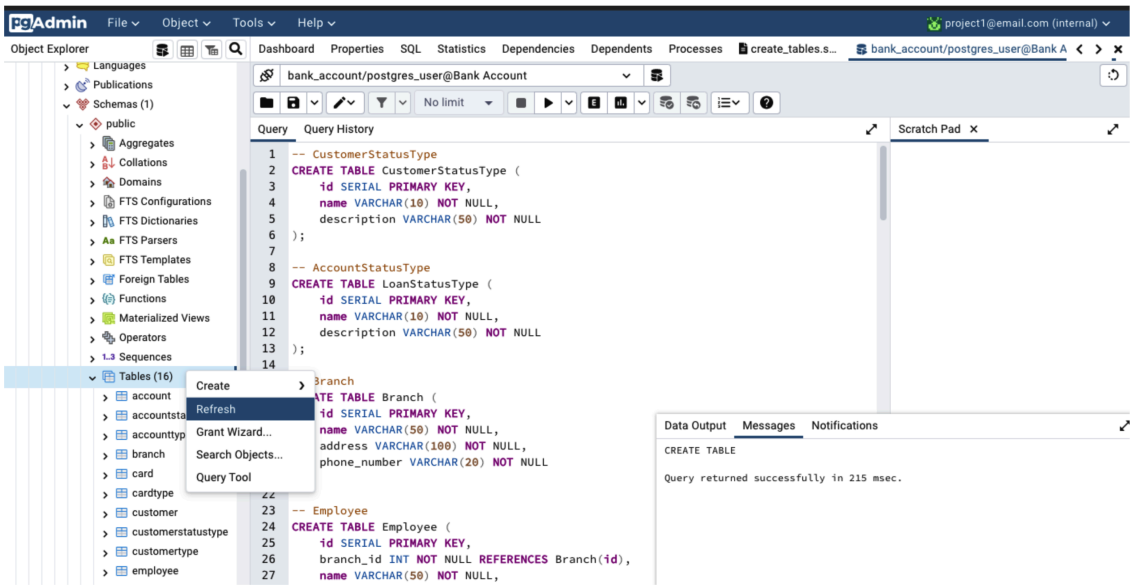

Figure 12-4. Creation and visualization of the tables

NOTE

In this project, you'll manually copy and execute SQL commands directly within pgAdmin's query console. This approach was intentionally chosen to familiarize you with the SQL syntax. It's worth noting that databases can also be interacted with programmatically. For example, in the upcoming projects, we'll utilize Python's database driver, psycopg2, to interact with PostgreSQL.

As you will see, 16 tables were created. To have an idea about our DDL queries, let's take a closer look at the `CREATE TABLE` statement for the account table:

```
-- PostgreSQL
CREATE TABLE Account (
    id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES Customer(id),
    branch_id INT NOT NULL REFERENCES Branch(id),
    type INT NOT NULL REFERENCES AccountType(id),
    currency VARCHAR NOT NULL,
    number VARCHAR NOT NULL UNIQUE,
    balance DECIMAL(10,2) NOT NULL,
    minimum_balance DECIMAL(10,2) NOT NULL DEFAULT 0,
    date_opened DATE NOT NULL,
    date_closed DATE,
    status INT NOT NULL REFERENCES AccountStatusType(id)
    CHECK (balance >= minimum_balance)
);
```

As you can see, the table has a serial ID as a primary key. This is used to automatically generate an increasing sequence of integers. Moreover, the table has four foreign keys referencing the IDs of four distinct tables: Customer, Branch, AccountType, and AccountStatusType. These keys are crucial for ensuring data integrity. Should, for example, a customer with ID 202 not exist in the customer table, the Customer ID foreign key will prevent the creation of an account associated with this customer. Lastly, a key feature of this table is the account balance, which specifies a minimum balance requirement for each customer. A check constraint is added to ensure the balance never falls below the customer-specific mini-

mum. Any attempt to update the balance to a value lower than the specified minimum will result in an error.

Now, let's populate the tables with some data. In SQL, it is common to use the term DDL to indicate statements that add, update, and delete records in a database. For most tables in our system, the insert operations are quite simple. For example, to add a new customer, you would need to run the following:

```
-- PostgreSQL
INSERT INTO Customer (
  customer_type_id, name, country, city, street,
  phone_number, email, status
) VALUES (
  1, 'John Smith', 'US', 'New York',
  '123 Main St', '123-456-7890', 'john@example.com', 1
);
```

The full script of the insert queries is available in this GitHub file. Copy all the insert queries, paste them into the Query Tool, and run the operation again. To see the data, right-click on a table name → View/Edit Data → All Rows. See Figure 12-5 for an illustration.

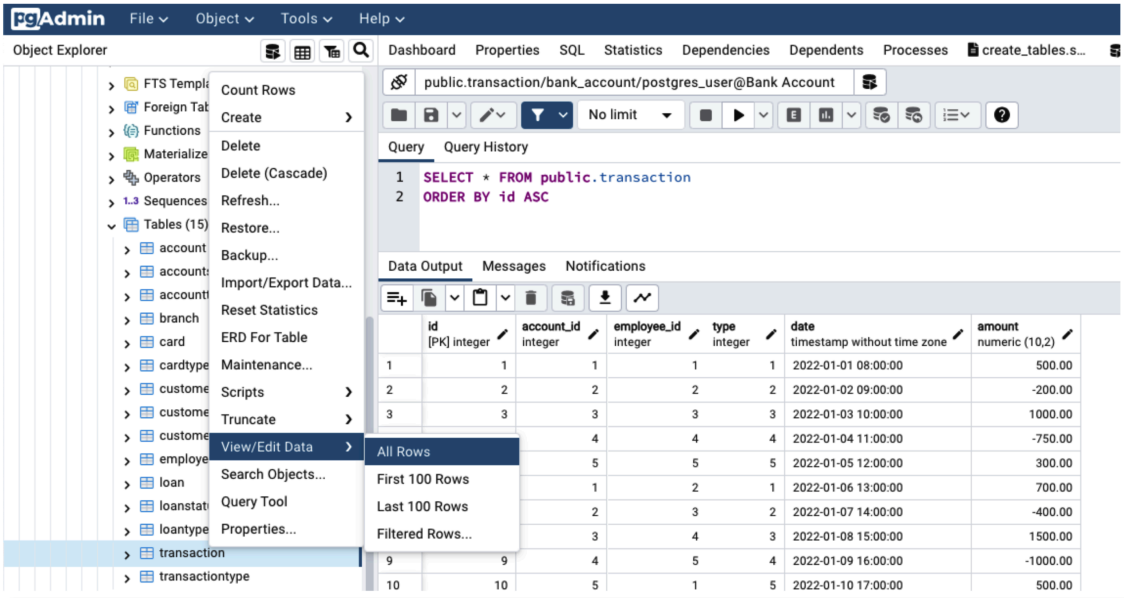

Figure 12-5. Visualization of table rows

Interestingly, some insert queries consist of multiple statements. For example, a payment transaction needs to be recorded in the transactions ta-

ble and requires a concurrent update of the account balance in the account table. To this end, both statements need to be wrapped with an explicit SQL transaction. It is called explicit because PostgreSQL executes any statement by default within an implicit transaction. Here is an example where a transaction of 40 USD is recorded for the account with ID = 1:

```
-- PostgreSQL
BEGIN;
INSERT INTO transaction (account_id, type, currency, date, amount)
VALUES (1, 4, 'USD', '2022-01-01 08:00:00', -40.00);
UPDATE account
SET balance = balance - 40.00
WHERE id = 1;
COMMIT;
```

Last but not least, an even more complex operation is the loan payment. Here, we need to record the payment in the loan payment table, then store it as a transaction in the transaction table, and finally update the customer's balance in the account table. To illustrate, here is an example of a loan payment of 1,000 USD recorded for the account with ID = 1:

```
-- PostgreSQL
BEGIN;
DO $$
DECLARE
    payment_amount DECIMAL := 1000.00;
BEGIN
    WITH inserted_transaction AS (
        INSERT INTO Transaction (account_id, type, currency, amount)
        VALUES (1, 1, 'USD', -payment_amount)
        RETURNING id
    )
    INSERT INTO LoanPayment (
      loan_id, transaction_id, payment_amount, scheduled_payment_date,
      payment_date, principal_amount, interest_amount, paid_amount
    )
    SELECT
      1, id, payment_amount, '2022-04-01', '2022-04-01',
      900.00, 100.00, payment_amount
    FROM inserted_transaction;
```

```
    UPDATE account
    SET balance = balance - payment_amount
    WHERE id = 1;
END$$;
COMMIT;
```

To create more transactions, run the queries in this GitHub file. The query for creating a loan payment is available in this book's GitHub repo.

Project 1: Clean Up

Once you are done with the project and you want to move to the next one, make sure you run the following command in the same root directory of Project 1:

```
# Bash
docker compose down
```

This command will stop and remove containers, networks, volumes, and images created by `docker compose up`.

Project 1: Summary

The goal for this project was to familiarize you with practical data modeling, database design, DDL and DML (Data Manipulation Language), and multistatement transactions. While this project provided a foundational understanding, it's important to note that a real-world bank account management system demands more extensive development. For example, implementing database security features like user roles and permissions, authentication, and data encryption will be necessary. Moreover, you will need a database optimization strategy that involves indexing, clustering, and typing. Finally, you will need to establish a reliable integration between the database and your application. This involves using database connectors, clients, connection pooling, object-relational mappers (e.g., SQLAlchemy), and more. Tackling all of these features is beyond the scope of this book and would require a lot more detailed explanation.

Project 2: Designing a Financial Data ETL Workflow with Mage and Python

In this project, you will design and implement a financial data ETL workflow. This involves retrieving historical stock price data from a financial API, applying various transformations, and ultimately storing the processed data in a database.

More specifically, we will be fetching the intraday open, close, high, and low prices, alongside trading volume, for four prominent stocks: Google, Amazon, IBM, and Apple. The source of our data will be the free version of the Alpha Vantage Stock Market API. Subsequently, we'll aggregate the intraday values to derive daily averages. Both the raw data and the transformed daily data will be stored in a PostgreSQL database. Let's assume that the workflow must be run once at the start of each month to retrieve data for the previous month.

Project 2: Workflow Definition

This project's workflow will have a linear structure consisting of the following steps:

1. Data retrieval
    1. Fetch adjusted intraday time series history for the past month using the `TIME_SERIES_INTRADAY` endpoint. As the free API version allows for 25 requests per day, we'll query data for 4 stocks: Amazon, IBM, Apple, and Google. The API request specifies the following parameters:
        1. A one-minute time interval between consecutive data points.
        2. `adjusted=true` to retrieve a time series adjusted by historical split and dividend distributions.
        3. The month parameter is set to the past month for monthly data retrieval.
        4. `outputsize=full` to obtain the full intraday history of the specified month.
        5. `datatype=csv` to receive the time series as a CSV file.

2. Raw data storage
    1. Store the raw data in the database, recording the ingestion timestamp.
3. Data aggregation
    1. Select the required columns for the primary transformation task, which will involve aggregating intraday values to compute daily averages.
    2. Compute daily aggregates by averaging the open, close, high, low, and volume columns, grouped by date and ticker symbol.
4. Deduplication
    1. Select columns for deduplication to address the aggregation step's behavior of computing aggregates without row grouping, where each row receives the corresponding aggregate of its group.
    2. Deduplicate the data, retaining the aggregated columns
5. Data export
    1. Export the daily averages to the database for further analysis.

We will execute each one of these steps in a linear sequence.

Project 2: Database Design

In our database, we will need two tables: one to store the raw intraday data retrieved from the API and another to store the transformed daily data produced at the end of our workflow. Figure 12-6 illustrates the ERD of these tables.

IntradayAdjusted	
price_timestamp	TIMESTAMP
symbol	STRING
open_price	FLOAT
close_price	FLOAT
high	FLOAT
low	FLOAT
volume	INTEGER
ingestion_timetamp	TIMESTAMP

DailyAdjusted	
price_date	DATE
symbol	STRING
daily_open	FLOAT
daily_close	FLOAT
daily_high	FLOAT
daily_low	FLOAT
daily_volume	FLOAT
ingestion_timetamp	TIMESTAMP

Figure 12-6. ERD of the API stock data

Let's move ahead to test our workflow in action.

Project 2: Local Testing

To begin testing, the initial step is to claim your Alpha Vantage API key on the Alpha Vantage website. Follow the instructions you find on the page, and you will receive an API key that you need to keep as a secret and never share it with anyone else.

Once you have the API key, you will need to navigate to the project's main directory. As before, this can be done by typing in the following command in your terminal:

```
# Bash
cd {path/to/book/repo}/FinancialDataEngineering/book/chapter_12/project_2
```

Once you are in the main project directory, you will find an *.env* file with a variable called `ALPHAVANTAGE_API_KEY`. Without using quotes, you must assign this variable the API key you get from Alpha Vantage (i.e., `ALPHAVANTAGE_API_KEY=YOUR_API_KEY`).

After that, run the project containers with the following command:

```
docker compose up -d
```

Once completed, you will have three running containers:

- Mage running on *http://localhost:6789/*. Mage is an open source data pipeline tool renowned for its user-friendly UI, ease of use, and extensive feature set, making it an ideal option for ETL workflows.
- A PostgreSQL instance where we will store the data.
- The pgAdmin client running on *http://localhost:8080/*. As before, you will use pgAdmin to create the project tables and explore the outcome of your workflow data outputs.

To begin, let's create our tables. Open a tab in your browser and navigate to *http://localhost:8080/*. Log in with the `PGADMIN_DEFAULT_EMAIL` and

`PGADMIN_DEFAULT_PASSWORD` that you have in your project's *.env* file. Follow the same steps as you did in the first project to create a server, and from the left sidebar, open the database called *stock_data*, then navigate to the schema called *public*. From there, you should be able to open a query tool and execute the queries you find in the *create_tables.sql* file to create the tables.

Once the tables are created, it's time to move on to build our ETL workflow. Open another browser tab and navigate to *http://localhost:6789/*. You should be able to see Mage's overview page, as illustrated in Figure 12-7. From the top left, click on "+ New pipeline" and select "Standard (batch)." You will be asked to give your pipeline a name; let's call it *Adjusted Stock Data.* Once done, click on Create.

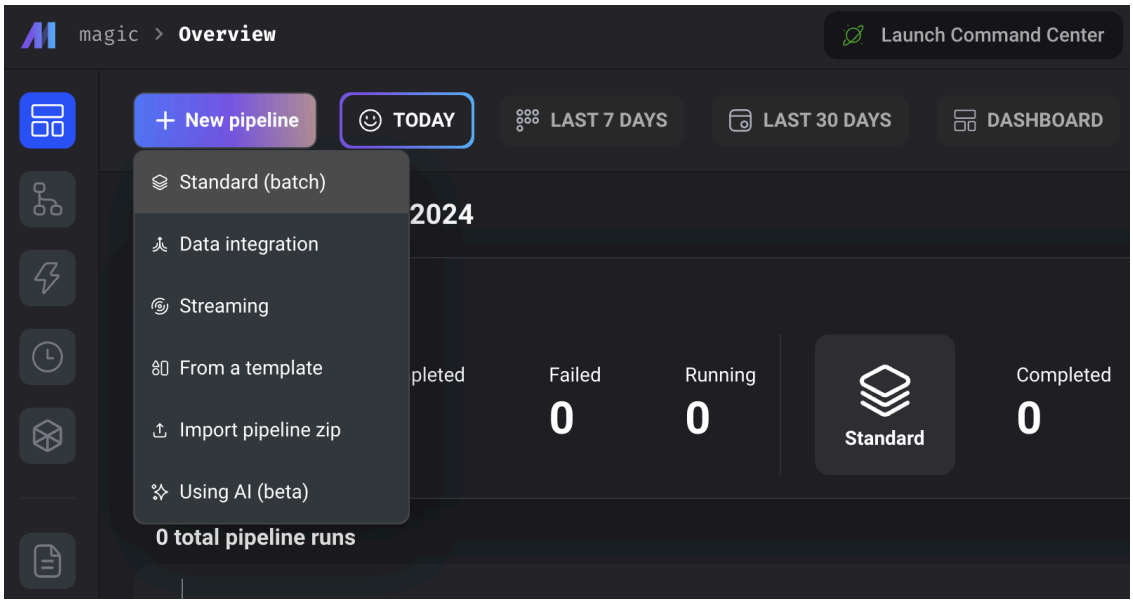

Figure 12-7. Mage overview page

After creating your pipeline, you will be redirected to the pipeline edit page, as shown in Figure 12-8.

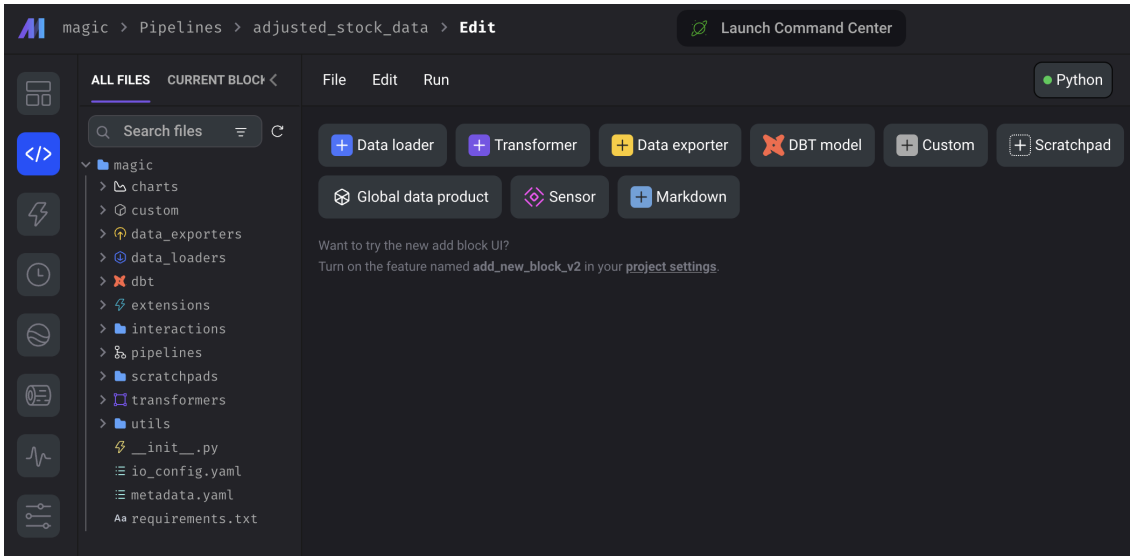

Figure 12-8. Pipeline edit page

From the left sidebar, open the file called *io_config.yaml*, delete all its content, and add the content of the *io_config.yaml* file to it. Once done, from the top menu, click File → "Save file" and then close the file editor. The *io_config* file is used by Mage to store and access credentials required to connect to databases and various storage systems.

After this, you will need to create the different components of our ETL workflow. From the same pipeline edit page (Figure 12-8), you can see different buttons that can be used to create an operator, such as data loader, exporter, and transformer. Let's create what we need as follows:

1. Create the API data loader:
    a. Click on +Data loader → Python → API.
    b. Name it *Load Data from Alpha Vantage* and then click "Save and add."
    c. In the code editor that appears, delete all code and replace it with the code in *fetch_intraday_data.py*.
    d. From the top menu, click File → Save pipeline.
    e. Use the small up arrow (^) on the top right of the data loader block to close it so you can better see the structure of your pipeline.
    f. In the subsequent phases, repeat steps d and e.
2. Create the raw data exporter:
    a. Click on +Data exporter → Python → PostgreSQL and call it *Export Raw Data*.

    b. Delete the content of the editor that appears and replace it with the code in *export_intraday_data.py*.
3. Create the aggregation column selection transformer:
    a. Click on +Transformer → Python → Column removal → Keep columns and name it *Select Aggregation Columns.*
    b. Paste the code from *select_columns_for_aggregation.py* into the editor.
4. Perform the aggregation:
    a. Click on +Transformer → Python → Aggregate → Aggregate by average value and name it *Compute Averages.*
    b. Paste the code from *compute_daily_aggregates.py* into the editor.
5. Create the deduplication column selection transformer:
    a. Click on +Transformer → Python → Column removal → Keep columns and name it *Select Deduplication Columns*.
    b. Paste the code from *select_columns_for_deduplication.py* into the editor.
6. Drop the columns that contain duplicates:
    a. Click on +Transformer → Python → Rows actions → Drop duplicates and name it *Drop Duplicates.*
    b. Paste the code from *drop_duplicates.py* into the editor.
7. Export the daily average data to the database:
    a. Click on +Data exporter → Python → PostgreSQL and call it *Export Daily Data.*
    b. Paste the code from *export_daily_data.py* into the editor.

Once these steps are done, your workflow is complete and ready to be executed. It should look like Figure 12-9.

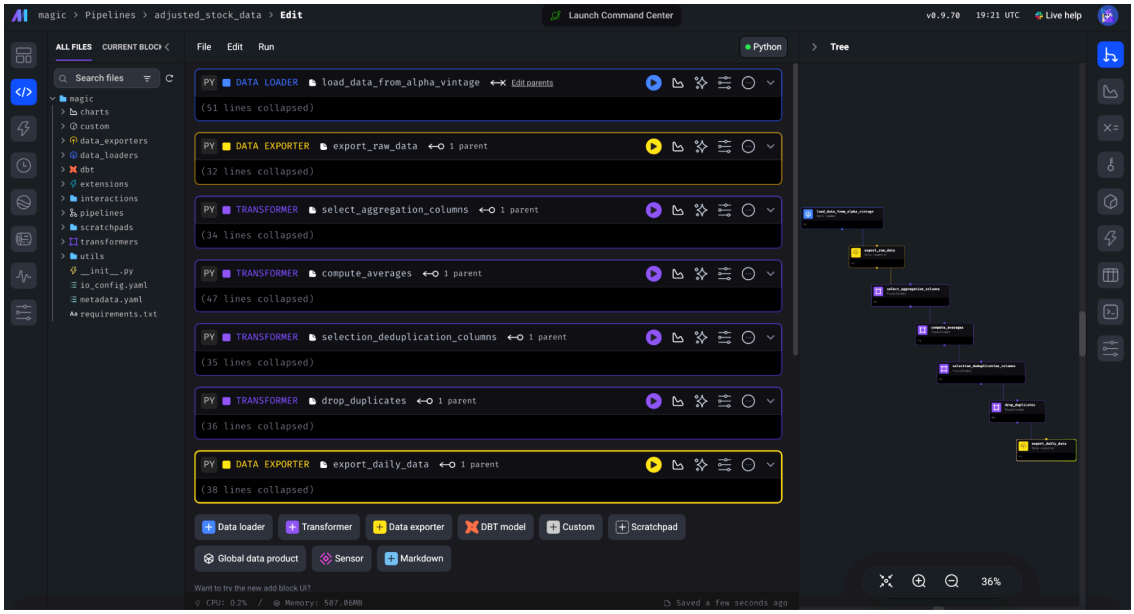

Figure 12-9. Overview of the full financial data pipeline

To perform a workflow execution test, navigate to the Pipelines section using Mage's left sidebar. Locate your pipeline named adjusted_stock_data. Click on the pipeline name, then select Run@once from the top menu, and confirm by clicking "Run now." A new execution line with a random name will appear in the table. Click on the name displayed, then patiently observe the execution progress as it advances through the seven steps.

Once completed with success, navigate to the pgAdmin tab in your browser and check the data in the database. You should see that both tables are populated with the data produced by your pipeline.

Project 2: Clean Up

Run the following command in the same root directory of Project 2:

```
# Bash
docker compose down
```

Project 2: Summary

Throughout this project, you've gained hands-on experience building a financial ETL workflow with Alpha Vantage API, Mage, Python, and

PostgreSQL. In real-world scenarios, you'll probably design and build similar pipelines. Yet, you will very likely need to incorporate more complex transformers and data quality validations. Additionally, you'll need to deal with challenges in pipeline management, such as scheduling, variable and secret management, triggers, scaling, concurrency, and data integration, among others. I strongly advise consulting the official Mage documentation to learn about these subjects and beyond.

Project 3: Designing a Microservice Workflow with Netflix Conductor, PostgreSQL, and Python

In this project, you will implement a microservice-based order management system (OMS) to process orders for an online store. Any online company that wants to streamline the entire order fulfillment process must have an OMS.

It's important to note that microservice workflows often entail more complexity than other types of workflows. Additionally, because microservices tend to be distributed in nature and deployed across diverse environments, accurately replicating microservice architectures locally can be quite challenging. Nonetheless, this project will offer a minimal example to provide insight into the structure and characteristics of microservices.

Project 3: Workflow Definition

The first step in designing a microservice workflow is to define its structure. To this end, let's start by outlining the microservices that will constitute our OMS system. Here are the five microservices we'll need:

Order acknowledgement service

Handles the acknowledgment of customer orders. It receives a client order, persists it in the database, and returns an acknowledgment message to the customer.

Payment processing service

Processes customer payment transactions and returns a message informing the customer about the status of their payment operation.

Stock and inventory service

Manages the inventory and stock levels of products available for sale. It tracks and checks the quantity of each product in stock, books and updates inventory based on incoming orders, and returns a message to the customer about the stock booking.

Shipping service

Manages the shipment of orders. This service coordinates with shipping carriers to book a delivery, generate a tracking number, and update the delivery status of orders. It returns a message informing the client about their upcoming delivery.

Notification service

Sends notifications to customers at various stages of the order fulfillment process.

As a next step, we need to define the dependency structure among our services. To keep things simple, we will design a linear workflow that executes one service after another. Once an order is submitted, it first gets acknowledged, then the payment gets processed, then the stock gets booked, and finally, a delivery is scheduled. Figure 12-10 illustrates the workflow.

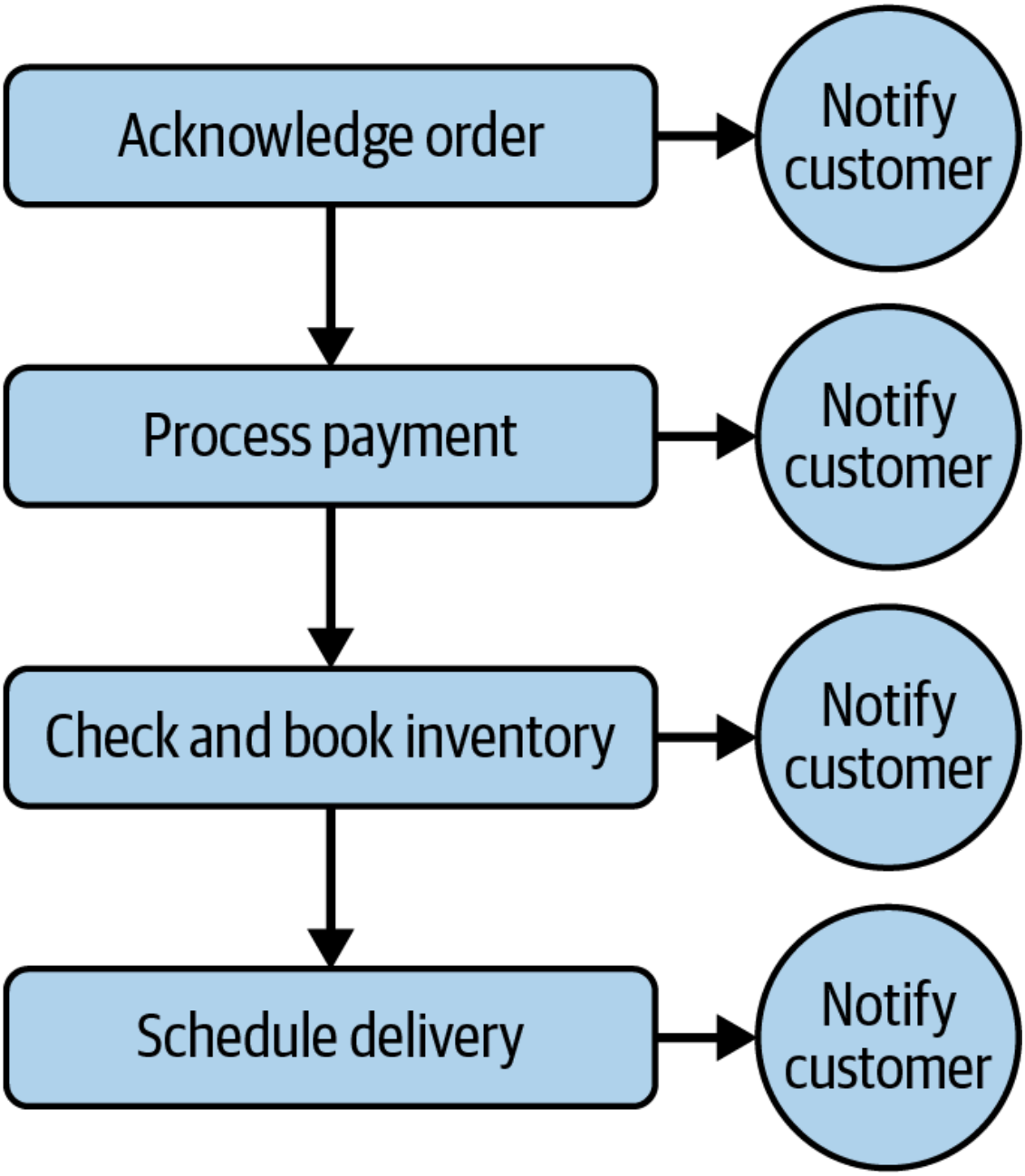

Figure 12-10. OMS workflow structure

Project 3: Database Design

A reliable database system is required to store and manage order-related data. In line with the other projects in this chapter, PostgreSQL stands out as our database solution of choice. Different database design patterns exist for microservices. While some advocate for a dedicated database per service, others favor a unified database shared across all microservices. To simplify our microservice structure, let's proceed with the latter approach—the single shared database pattern.

In our database, we will have five tables:

- The orders table stores orders along with their unique identifiers.
- The payments table stores transactional data related to order payments.
- The inventory table stores product inventory details.
- The stock bookings table stores the allocation of order items within the inventory.
- The delivery schedule table stores tracking delivery and shipment details.
- The notifications table stores customer notifications.

Figure 12-11 illustrates the ERD of our OMS database.

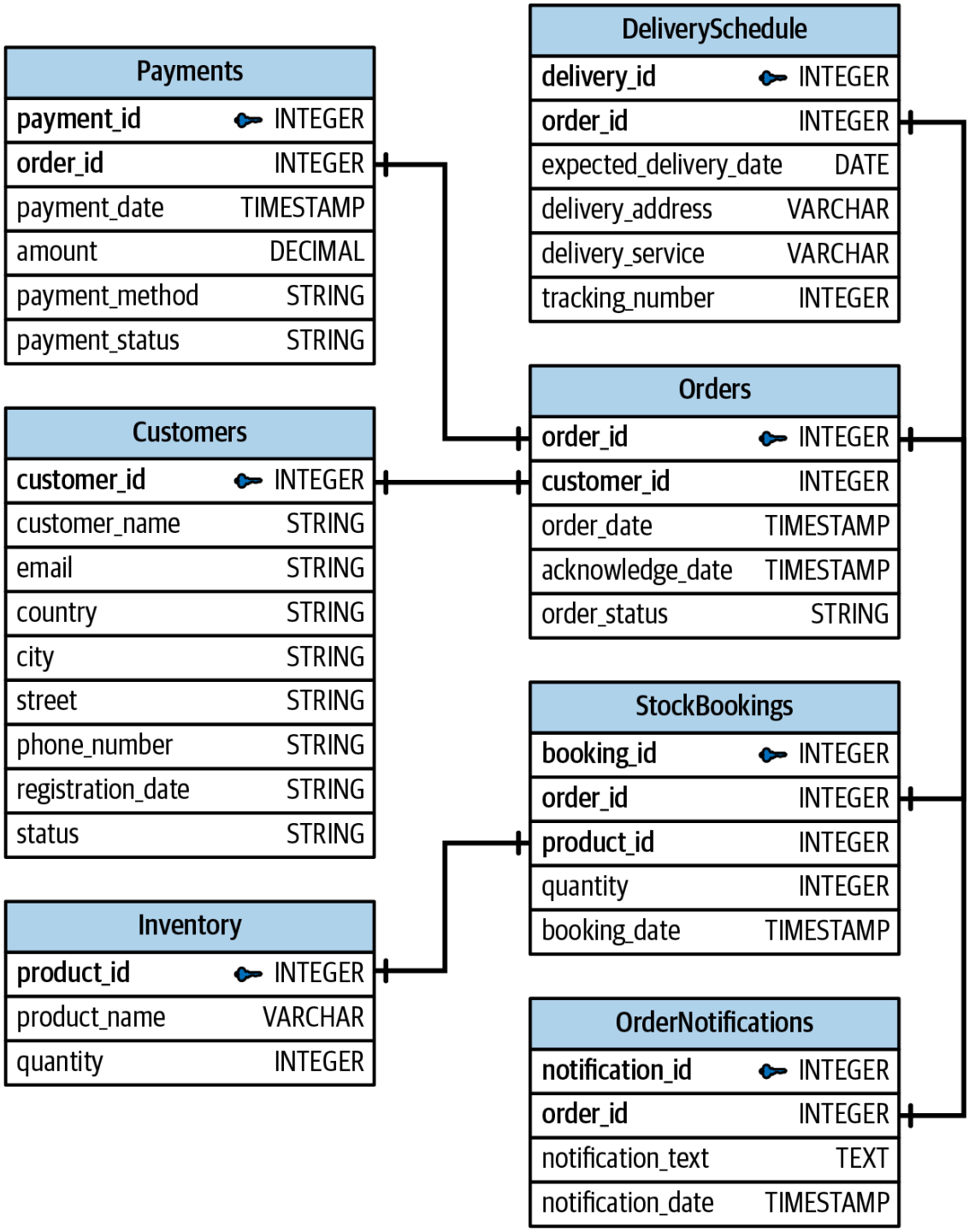

Figure 12-11. OMS database ERD

Next, let's move on to see our microservice system in action.

Project 3: Local Testing

As usual, you will need to navigate to the project directory in your terminal:

```
# Bash
cd {path/to/book/repo}/FinancialDataEngineering/book/chapter_12/project_3
```

Then, run our containers with the following command:

```
# Bash
docker compose up -d
```

Once completed, you will have three services running that we will be interacting with:

- JupyterLab to program and execute our workflow in Python. It will be running on *http://localhost:8888/*.
- Conductor orchestrator to explore and monitor workflows and executions via a user-friendly UI. It will be running on *http://localhost:5000/*.
- PgAdmin client UI to explore the data generated by executions. It will be running on *http://localhost:8081/*.

Open three browser tabs and paste the three localhost URLs into each. We will be interacting mostly with JupyterLab, so navigate first to the tab for *http://localhost:8888/*. Once it's ready, open a terminal from the Launcher tab. Consult Figure 12-12 for an illustration.

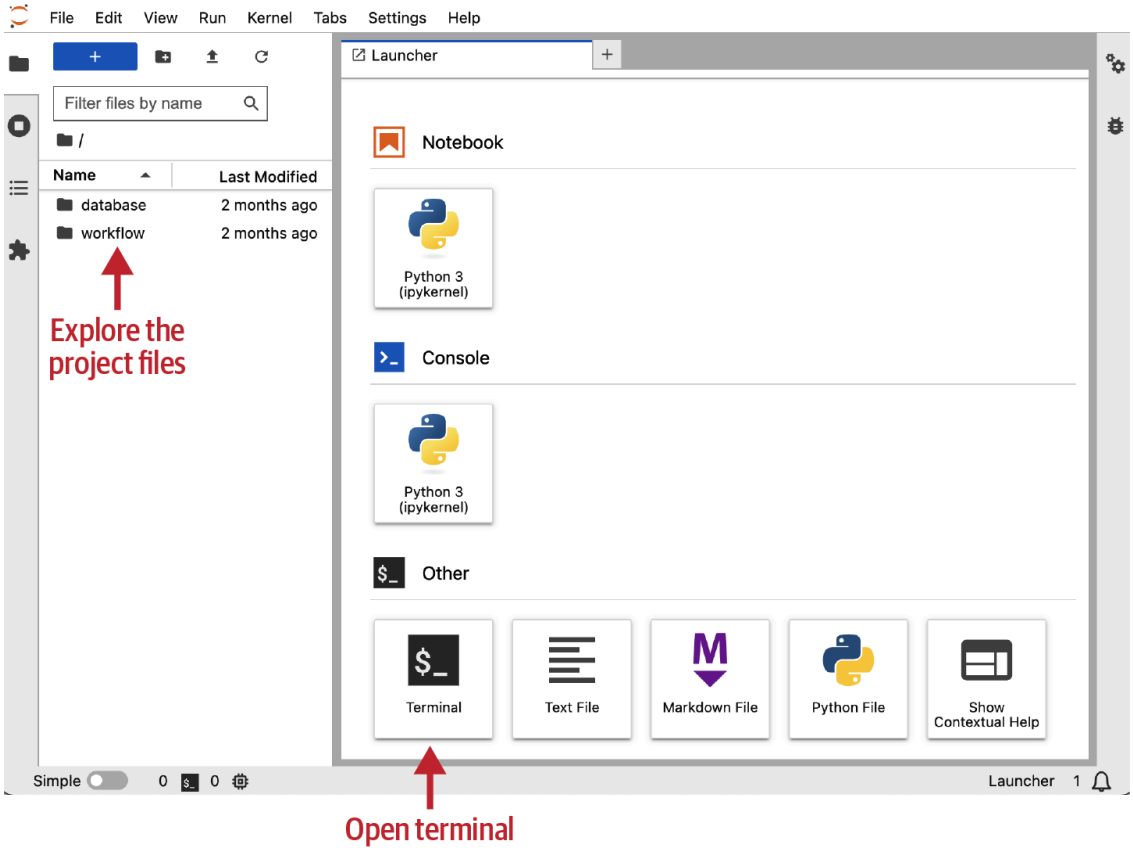

Figure 12-12. JupyterLab main overview page

In the terminal, type the following command and hit Enter:

```
# Bash
python3 database/create_tables.py
```

You should see a few logs that say “SQL executed successfully.” This means that the tables we want for our OMS have been created in Postgres.

Next, we want to insert some data into the customers and inventory tables. This is needed because, later on, we will be executing our workflow with orders that already contain the customer and product information. Again, in the terminal, type the following command, then press Enter to execute it:

```
# Bash
python3 database/populate_tables.py
```

The customer and product tables are now populated with data. If you wish to view the content of these files that you have just executed, you can navigate to the database folder using the right sidebar, as shown in Figure 12-12. If, in addition, you want to see the populated tables in the database, switch to the tab with *http://localhost:8081/* and log in with the `PGADMIN_DEFAULT_EMAIL` and `PGADMIN_DEFAULT_PASSWORD` values, which you can find in the *.env* file of the project. Once logged in, proceed with the same steps you learned in the first project to create a server and explore your tables.

The next step is to create and register our microservice workflow. First, we need to implement each mircoservice individually. This has been done for you, with each microservice residing within its respective folder within the *workflow* directory. You can use the left sidebar in JupyterLab to navigate to the *workflow* directory. You’ll find five folders in there, each corresponding to one of the previously defined microservices. Within each microservice folder, you’ll discover two scripts: one named *service.py*, which contains the microservice’s code, and the other named *worker.py*, containing the Conductor worker program for the respective

microservice. Each worker is tasked with executing a specific task, where the task corresponds to a microservice in our context.

NOTE

To keep things simple, the logic defined in each *service.py* contains a database insert/update operation. In real business problems, you will obviously need to include more functionality and business logic in your microservices. This may involve computations, transformations, API calls, database calls, and various other operations.

Moving to our workflow, to see how it is defined using Conductor, open the *order_workflow.py* file in the *workflow* directory from the sidebar. You will see how each task is defined by passing a reference name and the values for its parameters. The first task (acknowledge order) receives its input from the workflow input, while the other tasks take their input from the output of their previous tasks. More on passing input to tasks in Conductor is available in the Conductor documentation. Using Conductor, it is possible to create a linear flow using the >> operator. You can see how this is done at the bottom of the *order_workflow.py* file.

Now that our workflow is ready to be created, the next thing you can do is register the workflow in the Conductor database. To do so, go back to the Terminal screen in JupyterLab and paste the following command:

```
# Bash
python3 workflow/register_order_workflow.py
```

You should see a log message saying that the workflow has been successfully registered. To review your newly created workflow, go to the tab running *http://localhost:5000/*. There, you'll find the Conductor UI, which lets you check and monitor your workflow and task definitions and executions. Once there, use the top toolbar and navigate to the Definitions tab. You will find a table containing your workflows in your local instance. Click on the Order Workflow button to see the full details of the

OMS workflow you created. Figure 12-13 illustrates what this page looks like.

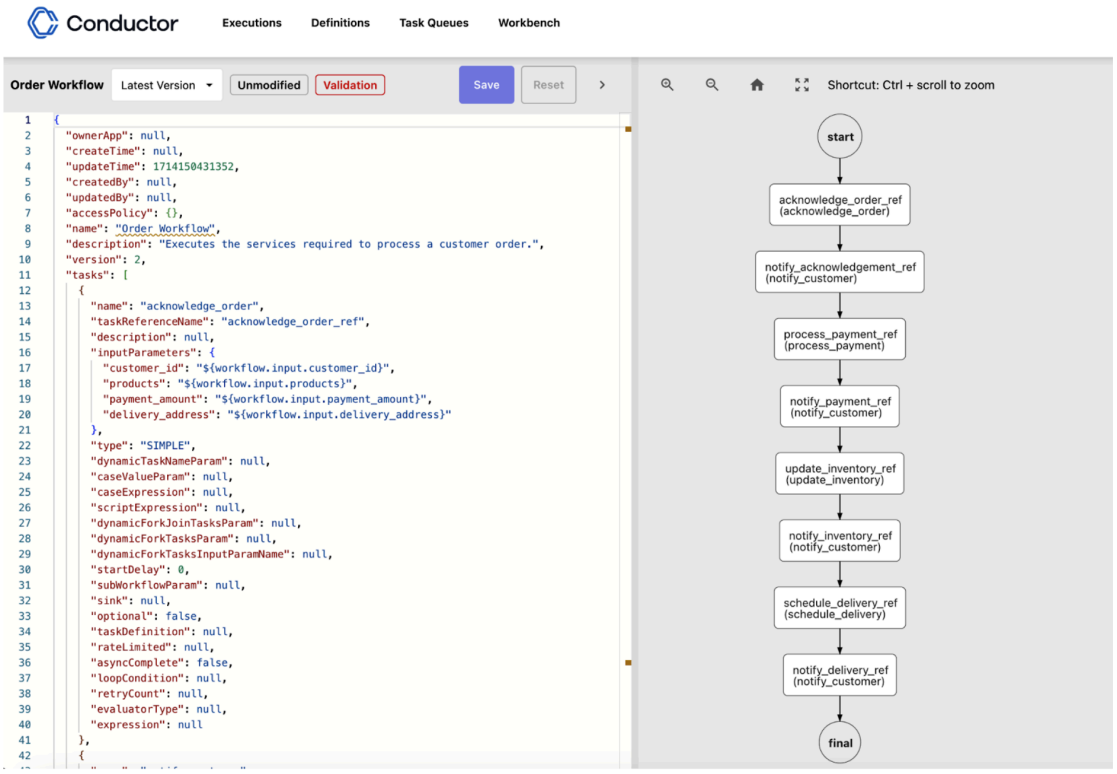

Figure 12-13. Order Workflow definition details page in Conductor

On the left side of Figure 12-13, you will see a lot of workflow definition parameters, many of which can be configured. To avoid extra complexity, we won't be setting any custom variables in this project. On the right side, you will see a visual representation of our OMS workflow. As you can see, it's a linear flow where tasks run in a sequence.

Now that we have our workflow registered, let's execute it. To do so, we need to run the file called *execute_order_workflow.py*, which you can find in the *workflow* folder. If you open this file, you will see that I already prepared a dummy workflow input with customer_id, products, payment amount, and delivery address. You can change these input values if you wish, but you need to make sure that the customer and products you add to the order exist in the database.

To execute our workflow, navigate to the Terminal tab in your JupyterLab and paste the following command:

```
# Bash
python3 workflow/execute_order_workflow.py
```

You should see a few log messages being printed that inform you about the successful outcome of the different microservices.

Now that you have executed the workflow, you can check the results in the database using pgAdmin. You'll find a record of your order in the orders table, payment details linked to your order in the payments table, updates in the inventory and stock bookings tables, and a booked shipment in the delivery table.

Finally, you can review the details of your workflow execution via the Conductor UI. Simply visit *http://localhost:5000/* and click on Executions from the top toolbar. In the table displayed on that page, you'll find a record corresponding to your workflow execution. Click on the ID listed under the workflowid column to access the details of this particular workflow execution. You should see a page similar to Figure 12-14.

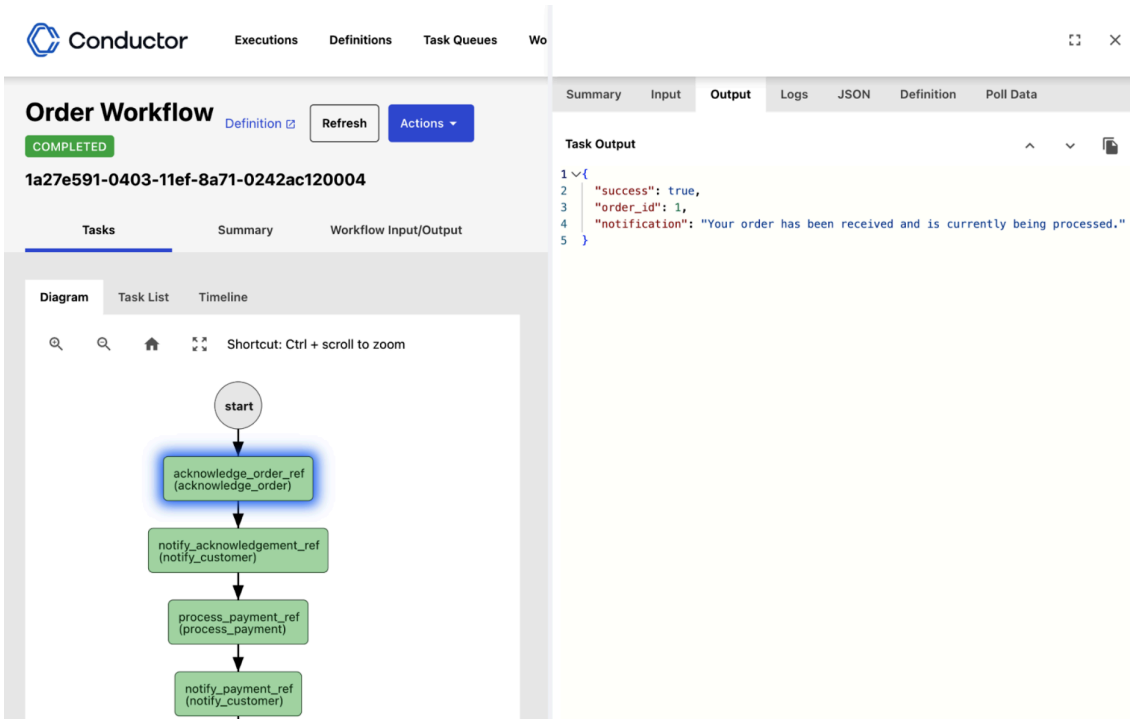

Figure 12-14. Order Workflow execution details page in Conductor

As shown in Figure 12-14, our workflow execution has completed successfully. By clicking on a specific task in the diagram on the left side, you can

view its execution details, including the input it received, the output it produced, its logs, and its task definition details.

Project 3: Clean Up

Run the following command in the root directory of Project 3:

```
# Bash
docker compose down
```

Project 3: Summary

This project aimed to familiarize you with the process of building and executing microservice workflows. In real-world situations, you'll likely tackle similar projects but with more complex requirements. For example, you are likely to deploy each of the services we defined into a separate environment, such as Cloud Functions or container-based deployment services, and configure security and access policies to allow them to communicate. Moreover, you will need to have a way to handle concurrency and dependencies in distributed transactions that span multiple services. Additionally, you might need to configure the workflow and task definitions to handle issues such as failure, retries, timeouts, etc.

Project 4: Designing a Financial Reference Data Store with OpenFIGI, PermID, and GLEIF APIs

In this project, you will develop a basic reference data store that holds information for identifying financial instruments and various entities, including the following:

- The full list of ISO 3166 Country Codes.
- The FIGIs, ISINs, Thomson Reuters Open Perm IDs, and RICs for the S&P 100 stocks. The S&P 100 is a primary stock index comprising 100

of the largest and most established companies listed on US stock exchanges.
- ISO 17442 Legal Entity Identifiers.
- ISO 10383 Market Identifier Codes.
- LEI-to-ISIN mappings.
- LEI-to-BIC mappings.

Throughout this project, you will be using three open APIs:

- OpenFIGI for retrieving FIGI data
- Open PermID for retrieving PermIDs and RICs
- GLEIF API for retrieving LEI and country code identifiers

If you are not familiar with financial identifiers, I strongly recommend reading Chapter 3 before beginning this project.

Project 4: Prerequisites

The only prerequisite for this project is to obtain a PermID API token. To do so, follow these steps:

1. Go to *https://permid.org*.
2. From the top right, click on REGISTER and follow the steps to complete your sign-up.
3. Once completed, go back again to *https://permid.org* and sign in with your login credentials.
4. Once signed in, from the top menu, click on APIs → Display my API Token (see Figure 12-15 for an illustration).

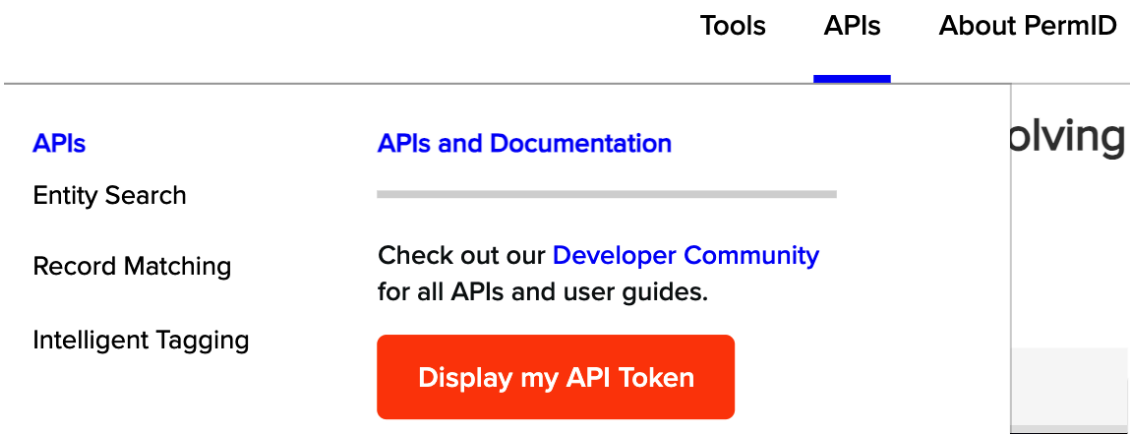

Figure 12-15. Finding your API token on PermID.org

In the following section, you'll use this token to communicate with the PermID API.

Project 4: Local Testing

To start, navigate to the project directory in your terminal:

```
# Bash
cd {path/to/book/repo}/FinancialDataEngineering/book/chapter_12/project_4
```

Once you are in the main project directory, you will find an *.env* file with a variable called `PERMID_API_KEY`. Without using quotes, you must assign this variable the API access token you got from PermID (i.e., `PERMID_API_KEY=YOUR_API_TOKEN`).

Run our containers with the following command:

```
# Bash
docker compose up -d
```

Throughout this project, you will be mainly interacting with JupyterLab notebooks. To begin, open your preferred browser and visit *http://localhost:8888/* to access JupyterLab.

The first thing we want to do is to create our tables in the database. These tables will store the various types of reference data that we will be load-

ing. From the main JupyterLab overview page, open a Terminal and execute the following command:

```
# Bash
python3 scripts/create_tables.py
```

To view the tables, log in to pgAdmin on *http://localhost:8081/* using the `PGADMIN_DEFAULT_EMAIL` and `PGADMIN_DEFAULT_PASSWORD` credentials you find in the *.env* file of the project. Once logged in, create a server with the `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD` credentials following similar steps as in Project 1. When done, you can check the tables using the left sidebar (Databases → reference_data → Schemas → public → Tables).

Now that you've created the tables, it's time to populate them. To accomplish this, you'll need to run multiple notebooks that have already been prepared for you. In JupyterLab, navigate to the *scripts* folder using the left navigation sidebar. Inside the scripts folder, you'll find several directories. Open the first one named *country*. Inside, locate a file named *country.ipynb*. Double-click to open it. Then, from the top menu, select Run → Run All Cells. This action will execute all the code cells in your notebook, fetching and uploading the ISO 3166 Country Codes from the GLEIF API.

Follow the same steps with these files:

- *scripts/figi/figi.ipynb*
- *scripts/lei/lei.ipynb*
- *scripts/lei_bic/lei_bic.ipynb*
- *scripts/lei_isin/lei_isin.ipynb*
- *scripts/mic/mic.ipynb*
- *scripts/permid/permid.ipynb*

After running all these scripts, our reference data store should be ready. Switch to the pgAdmin tab and examine the contents of the tables to observe the data in action.

Project 4: Clean Up

Run the following command in the same root directory as Project 3:

```
# Bash
docker compose down
```

Project 4: Summary

This project was designed to offer you an introduction to the world of financial reference data. Constructing and managing reference data stores are essential and common tasks in the financial industry. Although the reference data used in this project was simple and limited, handling reference data often presents much more complex challenges. For more information on this topic, refer to the section "Reference Data".

Conclusion

Congratulations on finishing the final chapter of the book! This is a significant milestone, and I want to thank you for your dedication and commitment to completing this journey.

Throughout the first 11 chapters of this book, you've gained knowledge about a range of foundational and finance domain-specific subjects essential for your journey as a financial data engineer. In this final chapter, you worked on hands-on projects crafted to solidify your understanding from earlier sections and refine your abilities through practical implementation.

Although this chapter concludes our journey through this book, your exploration of financial data engineering is just beginning. This field is continuously evolving with new trends that will shape the future of financial markets. In "The Path Forward: Trends Shaping Financial Markets", I'll briefly highlight these emerging trends, providing you with a glimpse of what lies ahead.

Follow Updates on These Projects

If the projects featured in this chapter undergo any updates or changes, I'll document them on the GitHub page. Feel free to refer to it if you encounter any issues.

Report Issues or Ask Questions

Should you encounter any challenges while setting up or executing any step in the projects outlined in this chapter, please don't hesitate to create an issue on the project's GitHub repository. I will make sure to reply to you in a very short time.